

1. Investigación técnica (Defensa TFG): arquitectura y pipeline operativo

1.1. 1) Alcance de este documento

Este informe **no** repite:

- docs/Personal Research/data-structure-and-canonical-schema-research-report.md
- docs/Personal Research/qrdqn-research-report.md

Se centra en los demás componentes clave del sistema para defensa ante tribunal.

1.2. 2) Arquitectura general del proyecto

El sistema está dividido en dos fases:

1. Fase 1 (offline entrenamiento/validación)

- Carga y limpieza de dataset (`src/load_cicids2017.py`)
- Entorno Gymnasium sobre dataset (secuencial) para decisiones binarias PERMIT/BLOCK (`src/rl_defender_env.py`)
- Entrenamiento (`src/train_rl_defender.py`)
- Validación A/B/C y leave-one-exact-CSV-out (`src/validate_checks.py`, `src/validate_le`)

2. Fase 2 (offline inferencia en tráfico real de laboratorio)

- Entrada mantenida: `scripts/predict_real_traffic_v2.py`
- Pipeline robusto con mapeo canónico, escalado, clipping y diagnósticos de shift.

1.3. 3) Procesado de datos (CICIDS2017) y anti-leakage

1.3.1. 3.1 Configuración de carga

La dataclass `CICIDSLoadConfig` controla rutas y preprocesado:

- `chunksize=250_000`
- `max_rows, sample_frac`
- `drop_identifier_cols=True`
- `scale=True`
- `use_canonical=True`
- `test_size=0.2, random_state=42`

Referencia: `src/load_cicids2017.py` (clase `CICIDSLoadConfig`, líneas ~41–62).

1.3.2. 3.2 Limpieza y saneado

Pipeline en `_prepare_cicids_features(...)`:

1. Detecta columna de etiqueta (`_find_label_column`)
2. Convierte etiqueta a binario (`BENIGN` $\rightarrow$ 0, resto $\rightarrow$ 1)
3. Elimina columnas con riesgo de leakage (`_drop_identifier_like_columns`): Flow ID, Timestamp, IPs, puertos, etc.
4. Fuerza numéricos (`_coerce_numeric_features`)
5. Sustituye `inf/-inf` por NaN y luego `NaN` $\rightarrow$ 0 en features (`_clean_rows`)
6. Elimina filas sin etiqueta

Referencias: `src/load_cicids2017.py` ($\sim$ 104–161, $\sim$ 221–272).

1.3.3. 3.3 Imputación y máscara de missingness

La imputación explícita es **relleno con 0** para features de flujo (`fillna(0)`). La semántica de máscara utilizada por el proyecto es:

- 1 = presente / válido
- 0 = ausente / imputado

Referencias:

- imputación: `src/load_cicids2017.py` ($\sim$ 150–160)
- semántica: `scripts/predict_real_traffic_v2.py` (`config["mask_semantics"] = "1=present,0=missing"`)

Nota para defensa: aquí conviene explicar que la imputación numérica y la máscara separan “valor” de “confianza de disponibilidad”.

1.4. 4) Entorno de RL usado en entrenamiento/validación

Clase: `RLDatasetDefenderEnv` (`src/rl_defender_env.py`).

1.4.1. 4.1 Interfaces Gymnasium

- `reset(...)` devuelve (`obs`, `info`)
- `step(action)` devuelve (`obs`, `reward`, `terminated`, `truncated`, `info`)
- `action_space` = `Discrete(2)` (0=PERMIT, 1=BLOCK)
- `observation_space` = `Box(shape=(n_features,), dtype=float32)`

1.4.2. 4.2 Recompensa

Config por defecto:

- `tp=1.5`
- `fp=-1.5`
- `fn=-5.0`
- `omission=0.0`

La función `_compute_reward(...)` penaliza fuertemente los falsos negativos (ataque permitido), lo que prioriza seguridad.

1.4.3. 4.3 Dinámica del episodio

- Avance secuencial sobre muestras
- `shuffle=True` opcional en entrenamiento
- fin por `terminated` (fin dataset) o `truncated` (`max_steps_per_episode`)

Referencia: `src/rl_defender_env.py` (`constructor`, `_compute_reward`, `step`).

1.5. 5) Fase 2: inferencia robusta en tráfico real

1.5.1. 5.1 Flujo operativo (main)

1. Lee CSV de flujos reales
2. Separa metadatos (`src_ip`, `dst_ip`, `protocol`, puertos, `timestamp`, y columnas `truth` si existen)
3. Armoniza unidades temporales (`maybe_convert_time_units`): segundos $\rightarrow$ micro-segundos si detecta mediana < 1
4. Mapea al espacio canónico (`map_to_canonical`)
5. Aplica clipping percentilar opcional en features crudas (`-percentiles`)
6. Aplica escalado (`-scaler`) salvo `-no-scale`
7. Aplica clipping z-score opcional (`-clip-z`)
8. Calcula diagnósticos de shift (`compute_diagnostics`)
9. Carga modelo (`load_model`) y predice por lotes (`batched_predict`)
10. Exporta artefactos en `runs/phase2/<RUN_ID>/`: `predictions.csv`, `config.json`, `metrics.json`, `diagnostics.json` (si `-export-diagnostics`)

Referencia: `scripts/predict_real_traffic_v2.py` (~141–475).

1.5.2. 5.2 Robustez ante outliers y drift

Funciones de `src/scaling_utils.py`:

- `apply_percentile_clipping(X, p_low, p_high)` (antes de escalar)
- `apply_z_clipping(X_scaled, max_z)` (después de escalar)

Objetivo: evitar que outliers de tráfico real empujen al modelo fuera del régimen de entrenamiento.

1.5.3. 5.3 Métricas en inferencia con truth opcional

`compute_truth_metrics(...)` calcula, si hay columnas de verdad:

- accuracy
- precision/recall/F1 de ataque
- TP/TN/FP/FN

Si no hay truth válida, la inferencia sigue y reporta métricas de distribución/predicción (`block_rate`, etc.).

1.6. 6) Mensaje clave para tribunal

La arquitectura está pensada para:

- entrenar y validar de forma controlada en Fase 1,
- ejecutar inferencia reproducible en Fase 2 con controles explícitos de drift (diagnósticos + clipping),
- manteniendo guardrails anti-leakage en la preparación de datos.